



The Lazy Developer

16

MODULE

Understanding APIs, how they can boost your app, and integrating them.

APIs (Application Programming Interfaces) are hugely common and can transform your apps ability to do so much more. Need to integrate with DeepSeek or ChatGPT? How about run a workflow and get some data? Maybe scrape another application? APIs help you handle all this with ease.

Table of Contents

01. What are APIs and how are they used? (20 min)
02. API banks like Apify and Hugging Face (20 min)
03. Integrating LLMs into your app (22 min)
04. Remember to manage to costs with rate limiting and batch calls. (22 min)
05. Consider your own API to access information in your app. (22 min)

What Are APIs and How Are They Used?

Imagine you're sitting in a restaurant. You look at the menu and decide what you want to eat. But you don't walk into the kitchen and start cooking it yourself. Instead, a waiter comes over, takes your order, walks it back to the kitchen, and then brings your food out when it's ready. You never see the kitchen. You don't need to know how the chef prepares the dish. You just place your order, wait a bit, and get exactly what you asked for.

That's essentially what an API does. API stands for Application Programming Interface, and it's the waiter between your app and some external service. Your app (the customer) makes a request, the API (the waiter) carries that request to the external service (the kitchen), and then brings back the response (your meal). The beauty of this is that you don't need to know how the external service works internally. You just need to know how to place your order correctly.

APIs are everywhere. When you check the weather on your phone, an API fetches that data from a weather service. When you pay for something online, an API talks to Stripe or PayPal to process your card. When you log in with Google, an API handles the authentication handshake. Once you understand APIs, you unlock the ability to plug your app into thousands of external services and make it do things that would take you months or years to build from scratch.

How APIs Actually Work (The Request-Response Cycle)

At the most basic level, an API works like a conversation between two computers. Your app sends a request to another server, and that server sends back a response. That's it. Every API interaction follows this pattern.

Let's break down the key parts of that conversation.

The URL (Where You're Sending the Request)

Every API has a base URL — think of it like the restaurant's address. For example, the OpenWeatherMap API lives at <https://api.openweathermap.org>. Then each specific piece of data you want has its own endpoint, which is like a specific dish on the menu. To get current weather data, you might hit:

```
https://api.openweathermap.org/data/2.5/weather?q=London&appid=YOUR_API_KEY
```

See how the URL contains the base address, the endpoint (`/data/2.5/weather`), and some parameters (`q=London` for the city, `appid` for your API key)? That's the structure of most API calls.

HTTP Methods (What You Want to Do)

When you make an API request, you also specify what kind of action you want to take. These are called HTTP methods, and there are four main ones you'll use constantly:

HTTP Method	What It Does	Restaurant Analogy	Real Example
GET	Retrieves data	Reading the menu	Fetch a list of products
POST	Creates new data	Placing a new order	Submit a contact form
PUT	Updates existing data	Changing your order	Update a user's profile
DELETE	Removes data	Cancelling your order	Delete a saved bookmark

The vast majority of API calls you'll make when starting out will be GET requests (fetching data from somewhere) and POST requests (sending data to somewhere). The other two become important when you're building your own API, which we'll cover later in this module.

Headers (The Extra Instructions)

Headers are like special instructions you give the waiter along with your order. They contain metadata about the request — things like what format you want the response in, your authentication credentials, or what type of content you're sending.

The most common headers you'll encounter are:

- **Content-Type:** application/json — Tells the server you're sending JSON data
- **Authorization:** Bearer YOUR_TOKEN — Sends your authentication token
- **Accept:** application/json — Tells the server you want JSON back

You don't need to memorise these. When you're working with an API, the documentation will tell you exactly which headers to include. And when you're using AI tools like Cursor or Claude Code, you can simply say:

```
Help me use OpenWeatherMap API with  
the correct headers and my API key so we can add weather data to our site.
```

The AI will handle the header configuration for you.

The Request Body (What You're Sending)

For POST and PUT requests, you'll often need to send data along with your request. This data goes in the request body, and it's almost always formatted as JSON (JavaScript Object Notation).

JSON looks like this:

```
{  
  "name": "Sarah",  
  "email": "sarah@example.com",  
  "plan": "premium"  
}
```

It's just a way of structuring data using key-value pairs. If you've ever filled out a form online, the data from that form typically gets converted into JSON before being sent to the server. JSON is the universal language that APIs use to pass information back and forth.

The Response (What You Get Back)

When the server processes your request, it sends back a response that includes:

- 1. A status code — A number that tells you whether things went well or not
- 2. Response headers — Metadata about the response
- 3. A response body — The actual data you asked for (usually in JSON)

Status Codes: How the Server Talks Back

Status codes are the server's way of saying "here's what happened with your request." You'll see these constantly when working with APIs, so it helps to know the common ones:

Status Code	Meaning	What It's Telling You
200	OK	Everything worked perfectly.
201	Created	Here's your data. Your new resource was created successfully.
400	Bad Request	You sent something wrong — maybe a missing field or invalid data.
401	Unauthorised	You didn't include valid authentication. Your API key might be wrong or expired.
403	Forbidden	You're authenticated but don't have permission to access this.
404	Not Found	The endpoint you're trying to reach doesn't exist. Check the URL.
429	Too Many Requests	You've hit the rate limit. Slow down.
500	Internal Server Error	Something broke on the server's side. Not your fault, but not helpful either.

The general rule is: 2xx codes are good (your request succeeded), 4xx codes mean you did something wrong (fix your request), and 5xx codes mean the server broke (wait and try again, or check the service's status page).

When you're debugging API issues, the status code is always the first thing to check. It tells you immediately whether the problem is on your end or theirs. So when you pass this to AI, they have a good starting point to figure out where else to start looking.

REST vs GraphQL (Keep It Simple)

You'll hear two main terms when people talk about API styles: REST and GraphQL. For your purposes right now, REST is what you'll use 95% of the time, but it helps to know the difference.

REST (Representational State Transfer) is the most common API style. Each piece of data has its own URL endpoint. Want users? Hit /api/users. Want a specific user? Hit /api/users/123. Want their orders? Hit /api/users/123/orders. It's straightforward and predictable.

GraphQL is a newer approach where you send a single request specifying exactly what data you want. Instead of hitting three different endpoints, you write one query that says "give me this user's name, email, and their last five orders." It's powerful but adds complexity.

Feature	REST	GraphQL
Learning curve	Easier to start with	Steeper learning curve
Number of endpoints	One per resource	Single endpoint for everything
Data fetching	May need multiple requests	Get exactly what you need in one request
When to use	Most projects, especially starting out	Complex apps with lots of related data
Tooling in Cursor	Excellent AI support	Good but less common in tutorials

For now, focus on REST. It's simpler, better documented, and what virtually every third-party API you'll integrate with uses.

API Keys and Authentication

Most APIs require you to identify yourself before they'll respond to your requests. This is done through API keys — unique strings of characters that act like a password for your app.

Here's the typical flow:

1. Sign up for the API service (e.g., OpenWeatherMap, Stripe, Brevo)
2. Generate an API key from your dashboard on their website
3. Store the key in your environment variables (.env file) — never in your code directly
4. Include the key in your API requests (usually in a header or query parameter)

This is critically important: never, ever put API keys directly in your frontend code. If your API key is in a JavaScript file that runs in the browser, anyone can open their browser's developer tools and see it. That means they can use your key, rack up charges on your account, or access data they shouldn't see.

The safe approach is to store keys in environment variables on your server:

```
# In your .env file (server-side)
WEATHER_API_KEY=abc123yourkeyhere
```

Then reference them in your server code:

```
// In your Express route
const apiKey = process.env.WEATHER_API_KEY;
```

If you need to call an API from your frontend, the proper pattern is to have your frontend call your own backend, and then your backend calls the external API with the key. This way the key never touches the browser.

Real-World API Examples

Let's look at some APIs you might actually use in your projects:

Weather Data — OpenWeatherMap

Want to show weather on your dashboard? OpenWeatherMap gives you current conditions, forecasts, and historical data. Free tier gives you 60 calls per minute.

Payments — Stripe

Need to accept payments? Stripe's API handles credit cards, subscriptions, invoices, and refunds. This is what The Lazy Developer platform uses for subscriptions.

Email — Brevo (formerly Sendinblue) or SendGrid

Want to send emails from your app? Brevo's API lets you send transactional emails (password resets, order confirmations) and marketing emails (newsletters). Again, this platform uses Brevo for its email notifications.

Authentication — Supabase Auth

If you've gone through the Supabase module, you're already using an API! Supabase's authentication system is accessed through their JavaScript client, which is really just a nice wrapper around their REST API.

AI — OpenAI, Anthropic, DeepSeek

Want to add AI capabilities to your app? These providers all offer APIs that let you send text and get intelligent responses back. We'll dive deep into this in the next section.

How to Read API Documentation

Every good API comes with documentation (often called "docs") that tells you exactly how to use it. API docs can feel overwhelming at first, but they all follow a similar structure. Here's what to look for:

1. Authentication section — How to get and use your API key
2. Endpoints list — All the available URLs you can hit and what they do
3. Request parameters — What data you need to send with each request
4. Response format — What the data looks like when it comes back
5. Code examples — Most docs include examples in multiple programming languages
6. Rate limits — How many requests you can make per minute/hour/day
7. Error codes — What specific error responses mean

Here's a great prompt to use when you're staring at API docs and feeling lost:

```
I'm looking at the API documentation for [service name].  
Here's the endpoint I want to use: [paste the endpoint].  
Can you explain what this endpoint does in plain English,  
what parameters I need to include, and write me a simple  
Express route that calls this API and returns the data  
to my frontend?
```

The AI will translate the docs into something actionable. This is one of the most powerful ways to use AI tools when you're learning — let them be your API documentation translator.

Testing APIs Before Writing Code

Before you wire an API into your app, it's smart to test it separately to make sure it works the way you expect. There are two popular tools for this:

Postman

Postman is a standalone desktop app specifically designed for testing APIs. You can:

- Enter any URL and choose the HTTP method
- Add headers and authentication
- Write request bodies in JSON
- See the full response including status code, headers, and body
- Save collections of requests for reuse

It's free for personal use and is the industry standard for API testing. If you're going to work with APIs regularly, Postman is worth installing.

Thunder Client (VS Code Extension)

If you'd rather stay inside your code editor, Thunder Client is a VS Code extension that does essentially the same thing as Postman but right inside your editor. Since you're likely using Cursor (which is built on VS Code), you can install Thunder Client from the extensions marketplace and test APIs without leaving your workspace.

Both tools let you experiment with API calls in a safe sandbox before committing to any code. Make a GET request, check the response, tweak the parameters, and only then write the code to integrate it into your app.

Putting It All Together: Your First API Integration

Let's walk through what a real API integration looks like from start to finish. Say you want to show weather data on your app's dashboard.

Step 1: Get your API key

Sign up at openweathermap.org and grab your free API key.

Step 2: Test in Postman or Thunder Client

Make a GET request to:

```
https://api.openweathermap.org/data/2.5/weather?
q=London&appid=YOUR_KEY&units=metric
```

Step 3: Ask AI to build the integration

```
Help me integrate the OpenWeatherMap API to show current
weather data on my dashboard. I want to display the
temperature, conditions, and an icon. My backend is
Express and my frontend is React. Store the API key
in an environment variable and make the API call from
the backend, not the frontend.
```

Step 4: Review and test

The AI will generate the backend route and frontend component. Test it locally, check the response, and adjust the display to match your design.

That's the pattern for every API integration: get the key, test it, prompt the AI to build it, review the code, and refine.

Key Takeaways

- An API is a messenger between your app and an external service. Your app sends a request, the API delivers it, and brings back the response. You never need to see the "kitchen."
- HTTP methods define the action: GET (read), POST (create), PUT (update), DELETE (remove). Most of your early work will use GET and POST.
- Status codes tell you what happened: 200 means success, 400 means you sent something wrong, 401 means bad authentication, 404 means wrong URL, 500 means the server broke.
- JSON is the universal format for sending and receiving data through APIs. It's just key-value pairs structured in a standard way.
- Never expose API keys in your frontend code. Store them in environment variables on your server and make API calls from your backend.
- REST is the API style you'll use most often. It's simpler and more widely supported than GraphQL, making it the right choice for beginners.
- Read the docs, but let AI translate them. API documentation can be dense. Paste the relevant sections into Cursor or Claude Code and ask for a plain-English explanation with code examples.

- Test before you build. Use Postman or Thunder Client to verify an API works the way you expect before writing integration code.
- APIs unlock superpowers for your app. Weather data, payments, email, AI, maps, authentication — instead of building these from scratch, you plug into existing services and focus on what makes your app unique.

API Banks Like Apify and Hugging Face

Imagine you need to renovate your kitchen. You could spend months learning plumbing, electrical work, and carpentry, then buy all the individual tools and materials yourself. Or you could walk into a massive home improvement store where everything is organised by category, pre-tested, and ready to go — plumbing aisle 3, lighting aisle 7, pre-built cabinets in the showroom. You pick what you need, follow the instructions, and your kitchen gets done in a fraction of the time.

That's what API marketplaces and platforms are. Instead of building complex functionality from scratch — web scraping, AI models, image processing, SMS messaging — you browse a catalogue of pre-built APIs, pick the one that fits your needs, plug it into your app, and move on. These platforms handle the heavy lifting so you can focus on what makes your app unique.

In this section, we'll explore the major API platforms you should know about, what they're best at, and how to evaluate whether a particular API is worth integrating into your project.

Apify — Web Scraping and Automation

If you've ever wanted your app to pull data from another website — product prices, social media posts, job listings, restaurant reviews — Apify is the platform you need to know about.

What Apify Does

Apify is a web scraping and automation platform that provides pre-built scrapers (called Actors) for hundreds of popular websites. Instead of writing complex scraping code yourself, you find an Actor that already does what you need, configure it, run it, and get structured data back.

How Actors Work

Think of an Actor as a little robot that visits a website, reads the pages like a human would, and then organises the information into clean, structured data (JSON) that your app can use.

Here's how the typical flow works:

1. Browse the Apify Store — Search for the website or data type you need
2. Choose an Actor — Each Actor has ratings, reviews, and usage stats
3. Configure it — Set parameters like which URLs to scrape, how many results, filters, etc.
4. Run it — Either manually, on a schedule, or triggered by your app via API
5. Get data back — Clean JSON data you can use immediately

Popular Apify Actors

Actor	What It Scrapes	Use Case Example
Instagram Scraper	Posts, profiles, hashtags, comments	Track brand mentions or gather content ideas
TikTok Scraper	Videos, profiles, trending content	Analyse viral trends for a content platform
YouTube Scraper	Video details, comments, channel data	Build a video analysis or summarisation tool
Amazon Scraper	Product listings, prices, reviews	Price comparison or market research app
Google Maps Scraper	Business listings, reviews, locations	Build a local business directory
Website Content Crawler	Any website's text content	Feed content to an AI for summarisation

Real-World Example

The Quiki.io platform (a video analysis tool built by the creator of this course) uses Apify extensively. When a user submits a TikTok or Instagram URL, an Apify Actor scrapes the video's metadata, captions, and engagement data. That data then gets sent to an AI for analysis. Without Apify, building custom scrapers for each platform would have taken weeks. With Apify, it was integrated in a day.

Pricing

Apify offers a free tier that includes \$5 of monthly usage credits — enough for testing and small projects. Paid plans start at \$49/month with higher usage limits. Each Actor also shows its estimated cost per run, so you can budget before committing.

Prompt example for getting started:

```
I want to scrape product prices from Amazon using Apify.
Help me set up the Amazon Product Scraper actor, configure
it to search for a specific product category, and write an
Express route that triggers the scraper and returns the
results to my frontend.
```

Hugging Face — AI and Machine Learning Models as APIs

If Apify is the place for web scraping, Hugging Face is the place for AI models. It's the largest open-source AI community in the world, hosting over 500,000 models that you can use through their API — no machine learning expertise required.

What Hugging Face Offers

Hugging Face hosts pre-trained AI models for virtually every AI task you can think of:

- Text generation — Generate articles, summaries, or creative writing
- Sentiment analysis — Determine if text is positive, negative, or neutral
- Text classification — Categorise content into topics
- Image classification — Identify objects in photos

- Image generation — Create images from text descriptions
- Translation — Translate between languages
- Speech-to-text — Convert audio to written text
- Question answering — Extract answers from documents

How to Use Hugging Face's Inference API

The simplest way to use Hugging Face models is through their Inference API. You don't need to download any models or set up any infrastructure. You just send a request and get a result:

```
// Example: Sentiment analysis using Hugging Face
const response = await fetch(
  'https://api-inference.huggingface.co/models/distilbert-base-uncased-finetuned-sst-2-english',
  {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${process.env.HUGGINGFACE_API_KEY}`,
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      inputs: 'I absolutely love this product!'
    })
  }
);

const result = await response.json();
// Returns: [{ label: 'POSITIVE', score: 0.9998 }]
```

Popular Model Categories

Category	Example Models	What You'd Build
Text Generation	GPT-2, Llama, Mistral	Chatbots, content generators
Sentiment Analysis	DistilBERT, RoBERTa	Review analysis, social media monitoring
Image Classification	ResNet, ViT	Photo tagging, content moderation
Translation	MarianMT, Helsinki-NLP	Multi-language app support
Speech-to-Text	Whisper	Transcription features, voice commands
Summarisation	BART, Pegasus	Article or document summarisers

Pricing

Hugging Face offers a generous free tier for their Inference API. Rate limits apply (around 30,000 requests per month for free), and you can upgrade to a Pro plan (\$9/month) for higher limits and faster response times. For serious production use, they offer dedicated inference endpoints starting at \$0.06 per hour.

RapidAPI — The Marketplace of Everything

RapidAPI is the largest API marketplace in the world, hosting thousands of APIs across every category imaginable. Think of it as the Amazon of APIs — whatever you need, someone has probably built an API for it.

Why RapidAPI Is Useful

The biggest advantage of RapidAPI is standardisation. Every API on the platform uses the same authentication method (a RapidAPI key), the same testing interface, and the same billing system. This means once you learn how to use one API on RapidAPI, you can use any of them.

Popular APIs on RapidAPI

API	Category	What It Does
API-Football	Sports	Live scores, standings, player stats
MovieDatabase	Entertainment	Movie details, ratings, trailers
Jokes API	Fun	Random jokes by category
Currency Exchange	Finance	Real-time exchange rates
Geolocation	Location	IP-based location detection
URL Shortener	Utility	Create short links programmatically

Pricing

RapidAPI itself is free to use. Individual APIs have their own pricing, and most offer a free tier with limited requests. You can test any API directly on the RapidAPI website before writing a single line of code.

Other Notable API Platforms and Services

Beyond the big three above, there are several specialised API services you should have on your radar:

Communication

- Twilio — Send SMS messages, make phone calls, and handle WhatsApp messages programmatically. If your app needs to send text message notifications or verification codes, Twilio is the go-to. Free trial includes a small credit balance.
- SendGrid — Email delivery at scale. Similar to Brevo but often preferred for high-volume transactional email. Generous free tier of 100 emails per day.

Media and Files

- Cloudinary — Upload, transform, and deliver images and videos. Automatically optimises file sizes, creates thumbnails, applies filters, and handles responsive images. Free tier includes 25,000 transformations per month.

- Uploadcare — Simple file upload API with built-in CDN. Great for user-generated content like profile pictures.

Maps and Location

- Mapbox — Beautiful, customisable maps for your app. More flexible and often cheaper than Google Maps. Free tier includes 50,000 map loads per month.
- Google Maps Platform — The industry standard for maps, directions, and place search. Free \$200 monthly credit.

Data and Content

- NewsAPI — Search and retrieve news articles from thousands of sources. Perfect for news aggregator features.
 - OpenAI, Anthropic, DeepSeek — LLM APIs for adding AI to your app (covered in detail in the next section).
-

How to Evaluate an API Before Integrating It

Not all APIs are created equal. Before you commit to integrating an API into your project, run through this evaluation checklist:

1. Documentation Quality

Can you understand how to use it within 10 minutes? Are there code examples? Is the documentation up to date? Poor documentation is a red flag — if you can't figure out how to use it, neither can the AI tools you'll be prompting.

2. Pricing and Free Tier

What's included for free? What happens when you exceed the free limit — do they charge you automatically, or does the API simply stop working? Always check for surprise billing.

3. Rate Limits

How many requests can you make per minute, hour, or day? Rate limits that are too low can cripple your app if it gets popular. Make sure the limits work for your expected usage.

4. Reliability and Uptime

Does the service have a status page? What's their historical uptime? An API that goes down frequently will make your app unreliable too.

5. Response Time

How fast does the API respond? If it takes 5 seconds to return weather data, your users will notice. Test response times during your evaluation.

6. Community and Support

Is there an active community? Can you find help on Stack Overflow or GitHub if something goes wrong? For paid tiers, what kind of support is available?

Free Tier Comparison Table

Platform	Free Tier	Best For	Rate Limit (Free)
Apify	\$5/month credits	Web scraping and automation AI/ML models	Varies by Actor
Hugging Face	30,000 requests/month		~1,000/day
RapidAPI	Varies per API	General API marketplace	Varies per API
Twilio	Trial credits (~\$15)	SMS and voice	Limited to verified numbers
Cloudinary	25,000 transformations/month	Image/video management	500 monthly transformations
Mapbox	50,000 map loads/month	Maps and location	50,000/month
SendGrid	100 emails/day	Email delivery	100/day
OpenWeatherMap	60 calls/minute	Weather data	1,000/day

Integrating an API Platform Into Your Project

Once you've chosen an API, the integration process typically follows these steps:

1. Sign up and get your API key
2. Read the quickstart guide (or paste it into Cursor and ask the AI to explain it)
3. Test the API using Postman, Thunder Client, or the platform's built-in tester
4. Store your API key in your server's .env file
5. Create a backend route that calls the API
6. Call your backend route from your frontend
7. Handle errors gracefully — what happens if the API is down or returns an error?

Here's a prompt template that works brilliantly for any API integration:

```
I want to integrate [API name] into my app. My backend is Express/Node.js and my frontend is React. Here's what the API documentation says about the endpoint I need: [paste relevant docs].
```

Please:

1. Create the Express route that calls this API
2. Store the API key in an environment variable
3. Create the React component that fetches data from my backend route
4. Handle loading and error states
5. Add appropriate error handling for common API failures

This prompt gives the AI everything it needs to build a complete, production-ready integration.

Key Takeaways

- API marketplaces are like home improvement stores for your app. Instead of building everything from scratch, browse pre-built solutions and plug them in.
- Apify is the go-to for web scraping. Its pre-built Actors can scrape social media, e-commerce sites, search engines, and more — saving you weeks of custom development.
- Hugging Face gives you access to 500,000+ AI models through a simple API. Sentiment analysis, text generation, image classification, translation — all available without machine learning expertise.
- RapidAPI is the largest API marketplace with a standardised interface. Once you learn how to use one API on the platform, you can use any of them.
- Always evaluate an API before committing. Check documentation quality, pricing, rate limits, reliability, and community support.
- The free tier is usually enough to start. Most platforms offer generous free tiers for testing and early development. Only upgrade when your usage demands it.
- Specialised services exist for almost everything: Twilio for SMS, Cloudinary for images, Mapbox for maps, SendGrid for email. Don't reinvent the wheel.
- Let AI tools help with integration. Paste API documentation into Cursor or Claude Code and ask for complete integration code. This is one of the most effective uses of AI-assisted development.

Integrating LLMs Into Your App

Think about the last time you used a really smart assistant — maybe you asked Siri to set a timer, or ChatGPT to explain something confusing. Now imagine embedding that same kind of intelligence directly into your own app. A product description that writes itself when you upload a photo. A support chat that actually understands questions. A content moderation system that reads user posts and flags anything inappropriate. That's what integrating an LLM (Large Language Model) into your app gives you.

Up until recently, adding AI to an application required a PhD in machine learning, a team of engineers, and a budget the size of a small country. Today? You make an API call. Seriously. The major AI providers — OpenAI, Anthropic, Google, DeepSeek, and Mistral — all offer APIs that let you send text in and get intelligent responses back. If you can make an API call (which you learned in the first section of this module), you can add AI to your app.

Let's walk through everything you need to know to make this happen.

The Major LLM Providers

Before diving into code, let's get the lay of the land. There are several major LLM providers, each with their own strengths, pricing, and personality. Here's a comparison to help you decide which one fits your project:

Provider	Key Models	Best For	Context Window	Approximate Cost (Input/Output per 1M tokens)
OpenAI	GPT-4o, GPT-4o-mini, o1	General purpose, widely adopted, great docs	Up to 128K tokens	\$2.50–\$15 / \$10–\$60
Anthropic	Claude Sonnet, Claude Opus, Claude Haiku	Long documents, nuanced reasoning, safety	Up to 200K tokens	\$0.80–\$15 / \$4–\$75
Google	Gemini 2.0 Flash, Gemini Pro	Multimodal (text + images), Google ecosystem	Up to 1M tokens	\$0.075–\$1.25 / \$0.30–\$5
DeepSeek	DeepSeek-V3, DeepSeek-R1	Cost-effective, strong reasoning, open weights	Up to 128K tokens	\$0.27–\$0.55 / \$1.10–\$2.19
Mistral	Mistral Large, Mistral Small	European provider, open-source options, multilingual	Up to 128K tokens	\$0.20–\$2 / \$0.60–\$6

Which should you choose? If you're just starting out, OpenAI is the safest bet — it has the best documentation, the largest community, and the most tutorials available. Anthropic's Claude is excellent for long-form content and complex reasoning tasks. DeepSeek offers remarkable quality at a fraction of the cost. Google Gemini shines when you need to process images alongside text. Mistral is a strong choice if you care about open-source or need strong multilingual support.

The good news is that switching between providers is relatively easy. The API patterns are very similar, so once you've integrated one, adding another is mostly a matter of changing the endpoint URL and adjusting the message format slightly.

Setting Up: API Keys and Environment Variables

Every LLM provider requires an API key. The signup process is nearly identical across all of them:

1. Create an account on the provider's platform
2. Navigate to the API section (usually under Settings or Developer)
3. Generate a new API key
4. Copy it immediately — most providers only show the key once
5. Add it to your server's .env file

```
# In your server/.env file
OPENAI_API_KEY=sk-proj-abc123...
ANTHROPIC_API_KEY=sk-ant-abc123...
DEEPSEEK_API_KEY=sk-abc123...
```

I cannot stress this enough: these keys must stay on your server. Never put them in your frontend code, never commit them to GitHub, never share them in a screenshot. If someone gets your API key, they can rack up charges on your account. OpenAI keys in particular are frequently scraped from public GitHub repos by automated bots.

Here's a prompt to help you set everything up:

```
I want to integrate the OpenAI API into my Express backend.
Help me:
1. Set up the environment variable for the API key
2. Install the openai npm package
3. Create a utility file that initialises the OpenAI client
4. Make sure the key is never exposed to the frontend
```

Making Your First LLM API Call

Let's look at what a basic LLM API call looks like. We'll use OpenAI as our example since it's the most common, but the pattern is very similar across all providers.

First, install the official SDK:

```
npm install openai
```

Then, in your Express backend:

```
// server/src/lib/openai.ts
import OpenAI from 'openai';

const openai = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY
});

export default openai;
```

```
// server/src/api/ai.ts
import express from 'express';
import openai from '../lib/openai';

const router = express.Router();

router.post('/generate', async (req, res) => {
  try {
    const { prompt } = req.body;

    const completion = await openai.chat.completions.create({
      model: 'gpt-4o-mini',
      messages: [
        {
          role: 'system',
          content: 'You are a helpful assistant that writes product descriptions.'
        },
        {
          role: 'user',
          content: prompt
        }
      ],
      temperature: 0.7,
      max_tokens: 500
    });

    const generatedText = completion.choices[0].message.content;
    res.json({ text: generatedText });

  } catch (error) {
    console.error('OpenAI API error:', error);
    res.status(500).json({ error: 'Failed to generate text' });
  }
});
```

Let's break down the important parts:

The Messages Array

LLM APIs use a messages array that represents a conversation. Each message has a role and content:

- **system** — Sets the overall behaviour and personality of the AI. Think of it as the job description you give the model before it starts working.
- **user** — The actual input from your user (or your app).
- **assistant** — Previous responses from the AI (used for multi-turn conversations).

The system message is incredibly powerful. By crafting a good system prompt, you can control the AI's tone, expertise, output format, and constraints. For example:

```
// A system prompt for a product description generator
{
  role: 'system',
  content: `You are an expert copywriter for an e-commerce store.
Write compelling product descriptions that are exactly 2-3 sentences.
Focus on benefits, not features. Use a friendly, conversational tone.
Never use exclamation marks or hyperbole.`
}
```

Temperature

Temperature controls how creative or deterministic the AI's responses are:

Temperature	Behaviour	Good For
0.0	Very deterministic — same input gives nearly the same output	Data extraction, classification, factual answers
0.3–0.5	Mostly consistent with slight variation	Customer support, summaries, translations
0.7	Balanced creativity and consistency	Product descriptions, blog posts
1.0+	Highly creative and unpredictable	Creative writing, brainstorming

For most practical applications, a temperature between 0.3 and 0.7 works well.

Max Tokens

Max tokens limits the length of the response. One token is roughly 3/4 of a word in English. So `max_tokens: 500` gives you roughly 375 words. Set this based on what you expect back — you don't want to pay for a 2,000-word essay when you only need a two-sentence description.

Streaming Responses

If you've used ChatGPT, you've seen how the text appears word by word rather than all at once. That's streaming, and it makes AI responses feel much faster and more interactive — even though the total generation time is the same.

Implementing streaming requires a bit more setup, but the user experience improvement is significant. Here's the basic pattern:

```
// Streaming endpoint
router.post('/generate-stream', async (req, res) => {
  try {
    const { prompt } = req.body;

    // Set headers for streaming
    res.setHeader('Content-Type', 'text/event-stream');
    res.setHeader('Cache-Control', 'no-cache');
    res.setHeader('Connection', 'keep-alive');

    const stream = await openai.chat.completions.create({
      model: 'gpt-4o-mini',
      messages: [
        { role: 'system', content: 'You are a helpful assistant.' },
        { role: 'user', content: prompt }
      ],
      stream: true // Enable streaming
    });

    for await (const chunk of stream) {
      const content = chunk.choices[0]?.delta?.content || '';
      if (content) {
        res.write(`data: ${JSON.stringify({ text: content })}\n\n`);
      }
    }

    res.write(`data: [DONE]\n\n`);
    res.end();

  } catch (error) {
    console.error('Streaming error:', error);
    res.status(500).json({ error: 'Streaming failed' });
  }
});
```

You don't need to implement streaming right away. Start with the simple non-streaming approach, get everything working, and then add streaming later as a polish step.

Practical Things You Can Build

Here are real features you could add to your app today using LLM APIs:

1. AI-Powered Description Generator

Let users upload a product photo and automatically generate a compelling description. The AI sees the image (if using a multimodal model like GPT-4o or Gemini) or takes a text description and crafts marketing copy.

```
Add an AI-powered description generator to my product page
using the Claude API. When a user clicks "Generate Description,"
send the product name, category, and key features to my backend.
The backend should call Claude with a system prompt optimised
for e-commerce copywriting and return a 2-3 sentence description.
```

2. Content Summariser

Got long articles or documents in your app? Let users click a button to get an AI-generated summary. This is perfect for news apps, research tools, or any platform with text-heavy content.

3. Smart Search

Instead of basic keyword matching, use an LLM to understand the intent behind a search query. A user types "something to keep my coffee warm" and the AI maps that to products like thermoses, heated mugs, and insulated cups.

4. Form Validation and Content Moderation

Use an LLM to check user-submitted content for quality, appropriateness, or completeness. For example, a job posting form where the AI reviews the description and suggests improvements before publishing.

5. Chatbot or Support Assistant

Build a chat interface that answers questions about your product using your own documentation as context. Feed the system prompt with your FAQ content and let the AI handle common queries.

Using Anthropic's Claude API

Since many builders in this community use Claude (both for coding with Claude Code and through the web interface), here's what the integration looks like with Anthropic's API:

```
npm install @anthropic-ai/sdk
```

```
import Anthropic from '@anthropic-ai/sdk';

const anthropic = new Anthropic({
  apiKey: process.env.ANTHROPIC_API_KEY
});

const message = await anthropic.messages.create({
  model: 'claude-sonnet-4-20250514',
  max_tokens: 1024,
  system: 'You are a helpful product description writer.',
  messages: [
    {
      role: 'user',
      content: 'Write a description for a handmade ceramic coffee mug.'
    }
  ]
});

console.log(message.content[0].text);
```

Notice how similar the structure is to OpenAI? The main differences are:

- The SDK import and initialisation
- The system prompt is a separate parameter (not in the messages array)
- The response structure is slightly different (message.content[0].text vs completion.choices[0].message.content)

But the core pattern — send messages, get a response — is identical.

Using DeepSeek's API

DeepSeek is particularly interesting because it offers strong performance at significantly lower prices. And here's a bonus: DeepSeek's API is compatible with the OpenAI SDK. That means you can use the same OpenAI npm package — you just change the base URL:

```
import OpenAI from 'openai';

const deepseek = new OpenAI({
  apiKey: process.env.DEEPSEEK_API_KEY,
  baseURL: 'https://api.deepseek.com'
});

const completion = await deepseek.chat.completions.create({
  model: 'deepseek-chat',
  messages: [
    { role: 'system', content: 'You are a helpful assistant.' },
    { role: 'user', content: 'Explain APIs in simple terms.' }
  ]
});
```

This OpenAI-compatible pattern is becoming an industry standard. Several other providers (including Mistral and many open-source model hosts) also support it, which makes switching between providers remarkably easy.

Important Considerations

Token Usage and Costs

Every API call costs money based on the number of tokens processed. Tokens include both your input (the prompt and system message) and the output (the AI's response). A rough rule of thumb: 1,000 tokens is about 750 words.

To keep costs under control:

- Use smaller models for simple tasks (GPT-4o-mini instead of GPT-4o)
- Keep system prompts concise
- Set appropriate max_tokens limits
- Cache responses for identical or similar queries (more on this in the next section)

Error Handling

LLM APIs can fail for various reasons — rate limits, network issues, content filters, or service outages. Always wrap your API calls in try-catch blocks and give your users a meaningful fallback:


```
try {
  const result = await generateWithAI(prompt);
  return result;
} catch (error) {
  if (error.status === 429) {
    // Rate limited — wait and retry
    return { error: 'AI service is busy. Please try again in a moment.' };
  }
  // Generic fallback
  return { error: 'AI generation failed. Please try again.' };
}
```

Content Filtering

All major LLM providers have content safety filters built in. If a user sends inappropriate content, the API will either refuse to respond or return a filtered response. You should still add your own moderation layer for user-facing features, but the built-in filters provide a good baseline.

Key Takeaways

- Adding AI to your app is just an API call. If you can make an API request, you can integrate an LLM. The hard part is deciding what to build with it, not the technical integration.
- Choose your provider based on your needs. OpenAI for general purpose and great docs, Claude for long-form and reasoning, DeepSeek for cost efficiency, Gemini for multimodal tasks.
- API keys must stay on your server. Never expose LLM API keys in frontend code. Always route requests through your Express backend.
- The messages array is the core of every LLM call. System messages set behaviour, user messages provide input, and assistant messages enable multi-turn conversation.
- Temperature controls creativity. Low temperature (0-0.3) for factual tasks, medium (0.5-0.7) for balanced content, high (0.8+) for creative tasks.
- Start simple, then add streaming. Get the basic request-response pattern working first. Add streaming later for a polished user experience.
- Most providers follow similar patterns. Once you've integrated one LLM API, switching to another is straightforward. DeepSeek even uses the OpenAI SDK directly.
- Watch your token usage. Use smaller models for simple tasks, set `max_tokens` appropriately, and cache responses where possible. We'll cover cost management in detail in the next section.

Remember to Manage Costs with Rate Limiting and Batch Calls

Picture this: you've just built a brilliant feature that uses the OpenAI API to generate product descriptions. You test it, it works beautifully, and you deploy it to production with a proud smile. Then you go to bed. While you sleep, a bot discovers your endpoint and hammers it with 50,000 requests. You wake up to an email from OpenAI: "Your usage this month: \$487.32." Your smile is gone.

This isn't a hypothetical. It happens all the time to developers who forget one critical step: managing API costs and protecting their endpoints. The same APIs that give your app superpowers can also drain your bank account if you're not careful. The good news is that preventing this is straightforward — you just need to know the patterns.

In this section, we'll cover everything you need to keep your API costs under control: understanding pricing models, rate limiting your own outbound requests, protecting your API from abuse, caching responses, batch processing, and monitoring your spend.

Understanding API Pricing Models

Before you can manage costs, you need to understand how you're being charged. Different APIs use different pricing models:

Pricing Model	How It Works	Examples
Per request	You pay a fixed amount for each API call	Weather APIs, geocoding APIs
Per token	You pay based on the amount of text processed (input + output)	OpenAI, Anthropic, DeepSeek
Per minute/hour	You pay for compute time used	Hugging Face dedicated endpoints
Tiered	Free up to a limit, then a fixed monthly fee for higher usage	Stripe, Brevo, most SaaS APIs
Per resource	You pay based on storage, bandwidth, or transformations used	Cloudinary, Supabase Storage

The most dangerous model for beginners is per-token pricing (used by all LLM providers), because costs can scale quickly if you're sending long prompts or receiving long responses. A single API call to GPT-4o with a large document might cost a few pence. But if your app sends that call 10,000 times a day? Now you're looking at hundreds of pounds per month.

Real Cost Examples

Let's make this concrete with some real numbers:

Scenario	API	Approximate Cost
1,000 product descriptions (500 words each)	GPT-4o-mini	~\$0.30
1,000 product descriptions (500 words each)	GPT-4o	~\$7.50
10,000 sentiment analyses	Claude Haiku	~\$0.80

10,000 sentiment analyses	Claude Opus	~\$75.00
100 web scrapes	Apify (basic actor)	~\$0.50
10,000 emails	Brevo (free tier)	Free
10,000 emails	SendGrid (beyond free tier)	~\$15.00

Notice the massive difference between model tiers. Using GPT-4o when GPT-4o-mini would do the job is like taking a limousine to the corner shop. Always use the smallest model that gets the job done.

Rate Limiting: Protecting Your App From the Outside

Rate limiting means restricting how many requests a user (or bot) can make to your API within a given time period. This is your first line of defence against abuse, accidental loops, and runaway costs.

Why You Need Rate Limiting

- Prevents abuse — Stops bots and bad actors from hammering your endpoints
- Controls costs — Limits how many expensive API calls can be triggered
- Ensures fairness — Prevents one heavy user from degrading performance for everyone else
- Protects upstream services — If your endpoint calls OpenAI, rate limiting your endpoint also rate limits your OpenAI spend

Implementing Rate Limiting with Express

The express-rate-limit package makes this incredibly simple:

```
npm install express-rate-limit
```

```
// server/src/middleware/rateLimiter.ts
import rateLimit from 'express-rate-limit';

// General API rate limiter
export const generalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15-minute window
  max: 100,                  // 100 requests per window per IP
  message: {
    error: 'Too many requests. Please try again in a few minutes.'
  },
  standardHeaders: true,
  legacyHeaders: false
});

// Stricter limiter for AI-powered endpoints
export const aiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15-minute window
```

```
// Apply to routes
import { generalLimiter, aiLimiter } from './middleware/rateLimiter';

// Apply general limiter to all routes
app.use('/api/', generalLimiter);

// Apply strict limiter to AI routes
app.use('/api/ai/', aiLimiter);
```

That's it. With about 15 lines of code, you've protected your entire API from being overwhelmed. The general limiter allows 100 requests per 15 minutes (plenty for normal usage), while the AI-specific limiter is much stricter at 20 requests per 15 minutes (because each AI call costs real money).

Here's a prompt to get this set up in your project:

```
Add rate limiting to my Express API. I want:
- A general limiter of 100 requests per 15 minutes per IP
  for all routes
- A strict limiter of 20 requests per 15 minutes for any
  routes under /api/ai/
- A very strict limiter of 5 requests per hour for any
  routes that trigger web scraping
- Return a clear JSON error message when limits are exceeded
- Use the express-rate-limit package
```

User-Based Rate Limiting

IP-based rate limiting is a good start, but for authenticated users, you might want to rate limit per user account instead. This prevents a single user from consuming your entire API budget:

```
// Rate limit by user ID instead of IP
export const userAiLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1-hour window
  max: 50, // 50 AI calls per hour per user
  keyGenerator: (req) => {
    return req.user?.id || req.ip; // Use user ID if logged in
  },
  message: {
    error: 'You have reached your hourly AI usage limit.'
  }
});
```

Rate Limiting Your Outbound Requests

The rate limiting above protects your API from incoming abuse. But you also need to think about rate limiting your outbound requests to external APIs. Most API providers enforce their own rate limits, and if you exceed them, they'll start returning 429 (Too Many Requests) errors.

For example, OpenAI's rate limits vary by plan:

OpenAI Tier	Requests Per Minute	Tokens Per Minute
Free	3	40,000
Tier 1 (\$5+ spent)	500	200,000
Tier 2 (\$50+ spent)	5,000	2,000,000
Tier 3 (\$100+ spent)	5,000	10,000,000

If your app tries to send 10 requests simultaneously on the free tier (which only allows 3 per minute), seven of those will fail. You need to throttle your outbound requests to stay within the provider's limits.

A simple approach is to use a queue with delays between requests. Libraries like p-queue or bottleneck make this easy:

```
npm install p-queue
```

```
import PQueue from 'p-queue';

// Process at most 3 requests per minute
const aiQueue = new PQueue({
  concurrency: 1,
  interval: 60000,
  intervalCap: 3
});

// Add requests to the queue
const result = await aiQueue.add(() =>
  openai.chat.completions.create({ ... })
);
```

Batch Processing: Do More With Less

Instead of making one API call per item, batch processing combines multiple operations into fewer calls. This saves money, reduces rate limit pressure, and often produces better results.

Example: Generating Descriptions in Batch

Instead of making 10 separate API calls for 10 products:

```
// BAD: 10 separate API calls = 10x the cost
for (const product of products) {
  const description = await generateDescription(product);
}
```

Combine them into a single call:

```
// GOOD: 1 API call for all 10 products
const prompt = `Generate a 2-sentence product description
for each of the following products. Return the results as
a JSON array.

Products:
${products.map((p, i) => `${i + 1}. ${p.name} - ${p.category}`).join('\n')}`;

const result = await openai.chat.completions.create({
  model: 'gpt-4o-mini',
  messages: [
    { role: 'system', content: 'Return only valid JSON.' },
    { role: 'user', content: prompt }
  ]
});
```

This approach turns 10 API calls into 1, cutting your cost by roughly 80-90% while still getting all the descriptions you need.

When Batching Works Best

Scenario	Without Batching	With Batching	Savings
10 product descriptions	10 API calls	1 API call	~85%
50 sentiment analyses	50 API calls	5 API calls (10 per batch)	~80%
100 content moderations	100 API calls	10 API calls (10 per batch)	~85%
Translating 20 strings	20 API calls	1-2 API calls	~90%

Be aware that batching has limits. If you try to stuff too much into a single prompt, you'll hit token limits or get lower-quality results. A good rule of thumb is to batch 5-15 items at a time, depending on how complex each item is.

Caching: Don't Pay Twice for the Same Answer

If a user asks your AI "What is an API?" and another user asks the exact same question five minutes later, do you really need to make a second API call? Of course not. Caching stores API responses so you can reuse them for identical or similar requests.

In-Memory Caching (Simple)

The simplest approach is to cache responses in your server's memory using a Map or a library like node-cache:

```
npm install node-cache
```

```
import NodeCache from 'node-cache';

// Cache with 1-hour expiry
const aiCache = new NodeCache({ stdTTL: 3600 });

async function generateWithCache(prompt) {
  // Create a cache key from the prompt
  const cacheKey = createHash('md5')
    .update(prompt)
    .digest('hex');

  // Check if we have a cached response
  const cached = aiCache.get(cacheKey);
  if (cached) {
    console.log('Cache hit – returning stored response');
    return cached;
  }

  // No cache – make the API call
  const result = await openai.chat.completions.create({ ... });
  const text = result.choices[0].message.content;

  // Store in cache for next time
  aiCache.set(cacheKey, text);

  return text;
}
```

This approach works great for single-server deployments. Every identical prompt will be served from cache after the first call.

Redis Caching (Production)

For production apps with multiple servers, Redis is the standard caching solution. It's an in-memory database that all your servers can share. We won't dive deep into Redis here, but know that it exists and is the next step when in-memory caching isn't enough.

Frontend Caching

Don't forget about caching on the frontend too. If your React app fetches the same data repeatedly, use TanStack Query (React Query) to cache API responses:

```
const { data, isLoading } = useQuery({
  queryKey: ['weather', city],
  queryFn: () => fetchWeather(city),
  staleTime: 5 * 60 * 1000, // Cache for 5 minutes
});
```

TanStack Query is already covered in other modules on this platform, and it's one of the most effective tools for reducing unnecessary API calls from your frontend.

Monitoring Usage and Setting Budget Alerts

Even with rate limiting and caching in place, you should actively monitor your API spend. All major providers offer usage dashboards:

- OpenAI: Dashboard at platform.openai.com/usage — set monthly spending limits
- Anthropic: Console at console.anthropic.com — view usage by model
- Stripe: Dashboard shows API call volume automatically
- Apify: Usage page shows credits consumed per actor

Setting Spending Limits

OpenAI allows you to set a hard cap on monthly spending. If you hit the cap, API calls simply stop working rather than running up a bill. This is the single most important thing you can do to prevent bill shock:

1. Go to platform.openai.com/account/billing
2. Set a monthly budget limit (e.g., \$20)
3. Set an email alert threshold (e.g., alert at \$10)

Other providers have similar features. Always set these up before you deploy to production.

Building Your Own Usage Tracking

For more granular tracking, log every API call in your database:

```
// Log every AI API call
async function trackApiUsage(userId, model, inputTokens, outputTokens) {
  await supabase.from('api_usage_log').insert({
    user_id: userId,
    model: model,
    input_tokens: inputTokens,
    output_tokens: outputTokens,
    estimated_cost: calculateCost(model, inputTokens, outputTokens),
    created_at: new Date()
  });
}
```

This lets you see exactly which users or features are consuming the most API credits, so you can optimise where it matters most.

Cost Comparison: Choosing the Right Tool for the Job

One of the easiest ways to cut costs is simply choosing the right model tier for each task. Here's a practical guide:

Task	Recommended Model	Why
Short product descriptions	GPT-4o-mini or DeepSeek-V3	Simple task, cheap models handle it well
Complex content generation	GPT-4o or Claude Sonnet	Needs higher quality reasoning
Sentiment analysis	Claude Haiku or GPT-4o-mini	Classification is easy for small models
Code generation	Claude Sonnet or GPT-4o	Needs strong reasoning for accuracy
Summarisation	DeepSeek-V3 or GPT-4o-mini	Good quality at low cost
Content moderation	GPT-4o-mini	Fast, cheap, consistent classification

The principle is simple: use the smallest model that consistently produces acceptable results for your specific task. Test with the cheaper model first. Only upgrade if the quality genuinely isn't good enough.

The Complete Cost Control Checklist

Here's your action plan for keeping API costs under control:

1. Set spending limits on every API provider dashboard before deploying
2. Implement rate limiting on all your Express endpoints, especially AI-powered ones
3. Use the smallest model that produces acceptable results for each task
4. Batch requests when processing multiple items — combine into fewer calls
5. Cache responses for identical or similar queries using node-cache or Redis
6. Throttle outbound requests to stay within provider rate limits
7. Monitor usage weekly — check dashboards and review your own usage logs
8. Set budget alerts so you know when costs are trending upward
9. Use frontend caching with TanStack Query to prevent redundant calls
10. Keep system prompts concise — long system prompts add token costs to every single call

Key Takeaways

- API costs can spiral quickly if you're not careful. A forgotten loop or an unprotected endpoint can result in hundreds of pounds in charges overnight. Always set spending limits.
- Rate limiting is essential. Use express-rate-limit to protect your endpoints from abuse. Apply stricter limits to expensive operations like AI generation.
- Batch processing saves 80-90% on costs. Instead of one API call per item, combine multiple items into a single request wherever possible.
- Caching prevents paying twice for the same answer. Use node-cache for simple deployments and Redis for production. Cache identical queries and set appropriate expiry times.

- Always use the smallest model that gets the job done. GPT-4o-mini handles most simple tasks just as well as GPT-4o at a fraction of the cost.
- Monitor and set alerts. Check your usage dashboards regularly and set email alerts before costs reach uncomfortable levels.
- Throttle outbound requests to stay within provider rate limits. Use queue libraries like p-queue to manage request flow.
- Your future self will thank you for setting up cost controls before you need them, not after you get the bill.

Consider Your Own API to Access Information in Your App

So far in this module, we've talked about consuming other people's APIs — calling OpenAI for AI responses, using Apify for web scraping, fetching weather data from OpenWeatherMap. But here's a question worth asking: what if other people (or other parts of your own system) want to access the data inside your app?

Think of it this way. You've built a restaurant and you've been ordering ingredients from various suppliers (external APIs). Now imagine someone wants to buy your signature sauce. Maybe a food truck wants to serve it. Maybe a catering company wants to include it on their menu. Maybe you want to sell it in bottles at the grocery store. To do any of that, you need a way to distribute your sauce — a supply chain of your own.

That's what building your own API does. It creates a structured, secure way for other systems to access the data and functionality inside your application. Your React frontend already talks to your Express backend — that IS an API. But in this section, we'll talk about designing it properly, so it's clean, consistent, secure, and ready for whatever comes next.

Why Build Your Own API?

You might be thinking: "My frontend already fetches data from my backend. Why do I need to think about this more carefully?" Great question. Here are the real-world scenarios where having a well-designed API matters:

1. Your Frontend Needs Structured Access

Even if the only consumer of your API is your own React app, a well-structured API makes development faster and debugging easier. Instead of scattered, inconsistent endpoints, you have a predictable pattern that both you and the AI tools you use can work with efficiently.

2. You Want to Build a Mobile App

If you ever want to create a mobile version of your app (using React Native, Flutter, or anything else), that mobile app will need to talk to the same backend. A clean API means the mobile team (or the AI helping you build it) can get started immediately without reverse-engineering your backend.

3. Third-Party Integrations

Maybe you want other apps to integrate with yours. Perhaps you're building a course platform and want to let other websites display your course catalogue. Or you're building a tool and want to offer a Zapier or Make.com integration. All of these need a public API.

4. Webhooks and Automation

When you want to trigger actions in other systems based on events in your app (like sending a notification when a new user signs up), you need well-defined API endpoints that external services can call.

5. Internal Microservices

As your app grows, you might split functionality into separate services. Your main app might talk to a separate AI processing service, or a separate notification service. They communicate through APIs.

REST API Design Basics

REST (Representational State Transfer) is the most common pattern for designing APIs, and it's what you'll use in Express. The core idea is simple: every piece of data in your app is a "resource," and each resource has a URL.

Resource-Based URLs

Good API URLs describe the resource they represent, not the action being performed:

Good (Resource-based)	Bad (Action-based)
GET /api/courses	GET /api/getCourses
GET /api/courses/123	GET /api/fetchCourseById?id=123
POST /api/courses	POST /api/createNewCourse
PUT /api/courses/123	POST /api/updateCourse
DELETE /api/courses/123	POST /api/removeCourse?id=123

The pattern is: the URL describes the thing, the HTTP method describes the action.

CRUD Operations Mapped to HTTP Methods

CRUD stands for Create, Read, Update, Delete — the four basic operations you can perform on any data. Here's how they map to HTTP methods:

Operation	HTTP Method	URL Pattern	Example	Description
Create	POST	/api/courses	Create a new course	Send course data in the request body
Read (all)	GET	/api/courses	List all courses	Returns an array of courses
Read (one)	GET	/api/courses/:id	Get a specific course	Returns a single course by ID
Update	PUT	/api/courses/:id	Update a course	Send updated data in the request body
Delete	DELETE	/api/courses/:id	Delete a course	Removes the course

This pattern repeats for every resource in your app. Users, courses, sections, bookmarks, orders — they all follow the same structure.

Building Express Routes: The Practical Structure

Let's build a real example. Say you're creating an API to manage courses and their sections. Here's how you'd structure it:

```
// server/src/api/courses.ts
import express from 'express';
import { supabaseAdmin } from '../lib/supabase';
import { authenticateToken } from '../middleware/auth';

const router = express.Router();

// GET /api/courses - List all courses
router.get('/', async (req, res) => {
  try {
    const { data: courses, error } = await supabaseAdmin
      .from('courses')
      .select('*')
      .order('order_index', { ascending: true });

    if (error) throw error;
    res.json({ courses });
  } catch (error) {
    console.error('Error fetching courses:', error);
    res.status(500).json({ error: 'Failed to fetch courses' });
  }
});

// GET /api/courses/:id - Get a specific course
router.get('/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { data: course, error } = await supabaseAdmin
      .from('courses')
      .select('*', { count: 'exact' })
      .eq('id', id)
      .single();

    if (error) throw error;
    if (!course) {
      return res.status(404).json({ error: 'Course not found' });
    }
    res.json({ course });
  } catch (error) {
    console.error('Error fetching course:', error);
    res.status(500).json({ error: 'Failed to fetch course' });
  }
});

// POST /api/courses - Create a new course (protected)
```

Then mount it in your main server file:

```
// server/src/server.ts
import courseRoutes from './api/courses';

app.use('/api/courses', courseRoutes);
```

Now every course-related operation has a clean, predictable URL. This is the pattern you'll repeat for every resource in your app.

Request Validation with Zod

Never trust data coming from the outside. Whether it's a user submitting a form or another service calling your API, you need to validate that the data matches what you expect. Zod is the most popular validation library in the TypeScript ecosystem.

```
npm install zod
```

```
import { z } from 'zod';

// Define a schema for course creation
const createCourseSchema = z.object({
  title: z.string()
    .min(3, 'Title must be at least 3 characters')
    .max(100, 'Title must be under 100 characters'),
  description: z.string()
    .min(10, 'Description must be at least 10 characters')
    .max(1000, 'Description must be under 1,000 characters')
    .optional()
});

// Use it in your route
router.post('/', authenticateToken, async (req, res) => {
  try {
    // Validate the request body
    const validatedData = createCourseSchema.parse(req.body);

    // If we get here, the data is valid
    const { data: course, error } = await supabaseAdmin
      .from('courses')
      .insert(validatedData)
      .select()
      .single();

    if (error) throw error;
    res.status(201).json({ course });
  } catch (error) {
    // Handle validation error
  }
});
```

Zod catches bad data before it ever reaches your database. If someone sends a request with a missing title or an empty string, they get a clear, helpful error message back instead of a cryptic database error.

Authentication: Protecting Your API with JWT

Not every endpoint should be accessible to everyone. Your API needs to know who is making the request and whether they're allowed to do what they're asking.

If you've gone through the Supabase and Security modules on this platform, you're already familiar with JWT (JSON Web Tokens). Here's a quick refresher on how it works in the context of your own API:

1. User logs in via Supabase Auth and receives a JWT token
2. Frontend stores the token and includes it in every API request (in the Authorization header)
3. Backend middleware verifies the token before processing the request
4. If valid, the request proceeds with the user's identity attached
5. If invalid, the request is rejected with a 401 Unauthorised response

The authentication middleware looks like this:

```
// server/src/middleware/auth.ts
export async function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({ error: 'Authentication required' });
  }

  try {
    // Verify the JWT token with Supabase
    const { data: { user }, error } = await supabaseAdmin
      .auth.getUser(token);

    if (error || !user) {
      return res.status(401).json({ error: 'Invalid token' });
    }

    req.user = user;
    next();
  } catch (error) {
    res.status(401).json({ error: 'Token verification failed' });
  }
}
```

Apply this middleware to any route that requires authentication. Public endpoints (like listing available courses) can skip it. Protected endpoints (like creating, updating, or deleting) should always use it.

Error Handling Patterns

Consistent error handling is what separates a professional API from a hobby project. Every error response should follow the same format so the frontend (or any consumer) always knows what to expect:

```
// Consistent error response format
{
  "error": "Human-readable error message",
  "code": "RESOURCE_NOT_FOUND", // Machine-readable error code
  "details": []                 // Optional: specific field errors
}
```

You can create a global error handler in Express:

```
// Global error handling middleware
app.use((err, req, res, next) => {
  console.error('Unhandled error:', err);

  // Default to 500 if no status was set
  const statusCode = err.statusCode || 500;
  const message = err.message || 'An unexpected error occurred';

  res.status(statusCode).json({
    error: message,
    code: err.code || 'INTERNAL_ERROR'
  });
});
```

The key principle is: never expose internal error details to the client. Your users should see "Failed to fetch course" not "relation \"courses\" does not exist" (which would tell an attacker about your database structure).

API Versioning

As your API evolves, you'll want to make changes without breaking existing consumers. The simplest approach is URL versioning:

```
/api/v1/courses    - Original API
/api/v2/courses    - Updated API with new features
```


This way, existing users continue using v1 while new features go into v2. In practice, most small to medium apps don't need versioning early on. But it's good to know it exists, and if you start with `/api/v1/` in your URL structure from day one, you'll be grateful later.

Testing Your API

Before (and after) building API endpoints, test them to make sure they work as expected. You have several options:

Postman or Thunder Client

As covered earlier in this module, these tools let you manually test each endpoint by sending requests and inspecting responses. Great for development and debugging.

cURL from the Terminal

For quick tests, you can use cURL directly from your terminal:

```
# GET all courses
curl http://localhost:8080/api/courses

# POST a new course (with authentication)
curl -X POST http://localhost:8080/api/courses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_JWT_TOKEN" \
  -d '{"title": "My New Course", "description": "A great course"}'
```

Automated Testing

For a more robust approach, you can write automated tests that run every time you make changes. Libraries like Jest and Supertest make this straightforward. You don't need to set this up immediately, but it's something to consider as your API grows.

When NOT to Build an API

Here's an important counterpoint: you don't always need a custom API. If the only consumer of your data is your own React frontend, and you're using Supabase, you might be able to query the database directly from the frontend using the Supabase client library.

Supabase's Row Level Security (RLS) ensures that users can only access the data they're authorised to see. For simple CRUD operations on user-specific data (like a user's bookmarks, preferences, or saved items), querying Supabase directly from the frontend is often simpler and faster than routing through your own API.

Build a custom API when:

- You need to combine data from multiple sources
- You're calling external APIs (that require secret keys)
- You need complex business logic that shouldn't live in the frontend
- Multiple clients (web, mobile, third parties) need to access the same data
- You need to enforce complex authorisation rules beyond what RLS provides

Use Supabase directly when:

- Simple CRUD on user-owned data
- Supabase RLS covers your access control needs
- You don't need to combine data from external sources
- Speed of development is the priority

The Lazy Developer platform itself uses a mix of both approaches — direct Supabase queries for simple data access and custom Express API routes for complex operations like subscription management, email notifications, and content administration.

Documentation: Swagger and OpenAPI

If you're building an API that other people (or other teams) will use, documentation becomes essential. OpenAPI (formerly known as Swagger) is the industry standard for documenting REST APIs. It creates interactive documentation where consumers can see every endpoint, the expected parameters, and even test the API directly from the browser.

Setting up OpenAPI is a more advanced topic, but know that tools like `swagger-jsdoc` and `swagger-ui-express` can auto-generate documentation from comments in your code. This is something to explore once your API is stable and you're ready to share it.

For now, a simple Markdown document listing your endpoints, parameters, and example responses is more than enough. You can ask the AI to generate this for you:

```
Look at all the route files in /server/src/api/ and generate
a complete API documentation in Markdown format. For each
endpoint, include: the URL, HTTP method, required headers,
request body format, response format, and example responses
for both success and error cases.
```

Putting It All Together: A Complete Prompt

Here's a comprehensive prompt that will generate a production-ready API resource:

Create a REST API with Express that exposes my app's courses and sections with the following requirements:

1. CRUD endpoints for courses (/api/v1/courses)
2. CRUD endpoints for sections (/api/v1/courses/:courseId/sections)
3. JWT authentication using Supabase on all write operations
4. Zod validation on all POST and PUT request bodies
5. Consistent error response format with error messages and codes
6. Rate limiting of 100 requests per 15 minutes
7. Proper HTTP status codes (200, 201, 400, 401, 404, 500)
8. Database queries using the Supabase Admin client

This single prompt gives the AI everything it needs to generate a well-structured, secure, validated API. Review the generated code, test each endpoint, and adjust as needed.

Key Takeaways

- Your Express backend is already an API. The question is whether it's well-designed, consistent, and ready for growth. Following REST conventions makes everything more predictable.
- Resource-based URLs are the standard. Use nouns for URLs (/api/courses) and HTTP methods for actions (GET, POST, PUT, DELETE). Avoid action-based URLs like /api/getCourses.
- CRUD maps cleanly to HTTP methods. Create = POST, Read = GET, Update = PUT, Delete = DELETE. This pattern repeats for every resource in your app.
- Always validate incoming data with Zod. Never trust data from the outside. Validate request bodies and return clear error messages when validation fails.
- Protect write operations with JWT authentication. Public read endpoints are fine, but creating, updating, and deleting data should always require authentication.
- Consistent error handling builds trust. Every error response should follow the same format. Never expose internal error details to the client.
- You don't always need a custom API. For simple CRUD on user-owned data, querying Supabase directly from the frontend with RLS is often simpler and faster.
- Start simple and grow. You don't need versioning, Swagger docs, or automated tests on day one. Get the basic CRUD working, add validation and authentication, and layer on the extras as your app matures.
- Let AI generate the boilerplate. API routes follow repetitive patterns that AI tools handle brilliantly. Describe what you need and let Cursor or Claude Code generate the scaffolding.